

Chapter 44: Ivy

1. Overview

Ivy is the name of Angular's third-generation compilation and rendering pipeline, shipped as the default in Angular 9 (with an opt-in preview in 8). It replaced **View Engine** (Angular's original renderer, dating back to Angular 2) and fundamentally changed how a `@Component` / `@Directive` / `@Pipe` / `@NgModule` decorator turns into executable JavaScript.

Ivy is not "a faster View Engine." It is a different compilation model built around three ideas:

1. **Locality** — the compiler can generate the final code for a component by looking *only* at that component's own decorator and template, never at the whole application's module graph.
2. **Instructions, not interpretation** — templates compile down to a sequence of plain function calls (`ɵɵelementStart`, `ɵɵtext`, `ɵɵproperty`, ...) that build and patch a real DOM incrementally, inspired by the "incremental DOM" technique pioneered for the same reason Ivy needed it: no per-app allocation of a virtual DOM tree.
3. **Tree-shakability by construction** — every piece of Angular's runtime that a component might need (change detection, DI, styling, i18n, animations renderer hooks) is referenced by that component's own compiled output as an ordinary imported function. If a component never uses `NgClass`-style multi-class bindings, or never uses i18n, the corresponding runtime code is never imported, so a bundler's tree-shaker can drop it.

The consequence reader should walk away with: Ivy turned Angular's compiler output from "one big monolithic factory graph per module" into "one small, self-describing, independently compilable artifact per component," and that single architectural shift is what enabled smaller bundles, faster rebuilds, better stack traces, dynamic component creation without `entryComponents`, and eventually the Ivy-only features (standalone components, `@if` / `@for` control flow, signals-based reactivity, hydration) that shipped in later majors and would have been very hard to retrofit onto View Engine.

2. Core Concepts

2.1 Why View Engine had to go

View Engine compiled templates into **NgFactory** files — one per component — but those factories were not self-contained. To generate `MyComponent.ngfactory.js`, the AOT compiler needed **whole-module, closed-world metadata**:

- Every component listed a `template / templateUrl`. To resolve `<app-child>` inside that template into a concrete class + factory call, the compiler needed to know which `NgModule` declared/imported `AppChildComponent`, because component→selector resolution happened at the **module** level, not the component level.
- `NgModule.entryComponents` existed *specifically* because View Engine could not know, just from a component's own metadata, whether it would ever be created dynamically (e.g., via `ViewContainerRef.createComponent` or a router-loaded lazy component). Anything dynamically created had to be pre-registered so the module-level compiler could emit a factory for it and put it in the `NgModuleFactory`'s component factory resolver.

- Compiling a single component therefore required type-checking and resolving the *entire* `NgModule` it belonged to, transitively touching every module that exported directives/pipes into scope. This is "whole-program" or "global" compilation.

Consequences of that design:

- **No tree-shaking at the component level.** The whole `NgModuleFactory` was one artifact; bundlers saw a large object graph of interdependent factories and could rarely prove any part was unused.
- **Large generated code.** `NgFactory` files contained verbose `ViewMyComponent_Host / ViewMyComponent_0` classes with imperative `createInternal / detectChangesInternal` methods, duplicating a lot of boilerplate per component (large "de-duplication surface" was missed).
- **Slow incremental builds.** Because compiling `ComponentA` could require re-resolving metadata for the whole module graph, editing one file in a big app could force much of AOT compilation to redo work. There was no clean unit of incremental compilation.
- **Dynamic component creation was clunky.** `entryComponents` was easy to forget, error messages were unhelpful ("No component factory found"), and library authors had to over-declare entry components defensively.
- **Poor debuggability.** Generated factory code was far removed from the template; stack traces pointed into synthetic `view_X_0.prototype.detectChangesInternal`, not into recognizable expressions.
- **i18n and metadata were bolted on** rather than expressed as first-class instructions, making extraction/translation pipelines more special-cased.

2.2 The Locality Principle

Locality is the constraint Ivy's compiler design imposes on itself: *generating the code for a directive/component/pipe must require only that class's own decorator metadata and its own template source — never information "reached into" from other files, other modules, or the global compilation unit.*

Concretely:

- The compiler does **not** need to know what module a component belongs to in order to compile the component itself. `@defineComponent` bakes the component's own selector, inputs, outputs, template instructions, and (for Ivy's component-level dependency resolution) a reference list of directives/pipes *the template can see* directly on the component's compiled definition (`directiveDefs / pipeDefs`, or lazily via a `dependencies` function for standalone components).
- Because a component "carries its own template-resolution table" instead of relying on a module compiled elsewhere, compiling `ComponentA.ts` never requires touching `ComponentB.ts`'s internals, `NgModuleEx.ts`, or anything else outside `ComponentA`'s own file (plus its statically-imported type references, which TypeScript already needs).
- This is what makes Ivy compilation **embarrassingly parallel and incremental**: the compiler (or a language service, or `ngtsc`) can compile/recompile exactly the one file that changed, re-emit its `.js` output, and be done — it never needs a second global "linking" pass across the whole program to decide what a component's factory looks like. (There *is* a lightweight "linker" step for library `.mjs` output using partial compilation, but that's a per-file partial-to-full IR transform, not a whole-program metadata resolution — see §5.)
- Locality is also why `entryComponents` disappeared: since a component's definition is self-sufficient the moment it's compiled, `viewContainerRef.createComponent(SomeComponent)` needs no pre-

registration — Ivy can synthesize a `ComponentFactory`-equivalent from `SomeComponent.ɵcmp` on the spot, at runtime, from the class reference alone.

Locality is the single idea from which almost every other Ivy benefit (tree-shaking, faster builds, better dynamic component API, simpler library packaging) is derived. If asked "what problem does Ivy actually solve," the correct one-sentence answer is: *it moves Angular from whole-program (global) compilation to per-file (local) compilation.*

2.3 Instruction-set / "incremental DOM" style codegen

Two competing strategies exist for compiling a template to executable code:

- **Virtual DOM (React-style):** the render function returns a tree of lightweight objects (a "vdom"); the framework diffs the new tree against the previous tree, then patches the real DOM. Cost: an allocation per node per render, plus a full diff.
- **Incremental DOM (Ivy's approach):** the compiled template *is* a sequence of imperative instruction calls that walk the real DOM structure directly — no intermediate tree is ever allocated. The "diffing" is implicit: on a re-render, the *same* instruction call happens at the *same* position, so the runtime instruction itself (e.g., `ɵadvance` + `ɵproperty`) can compare the new value to the value it stored last time (stored in `LView`, a flat array) and only touch the DOM if the value actually changed.

Ivy splits every template into two instruction blocks, generated by the compiler and stitched together in the component's `template` function, which the runtime calls twice per render pass with a `flags` parameter:

- **Create block** (`flags & RenderFlags.Create`): runs once, the first time the view is rendered. Emits `ɵelementStart`, `ɵtext`, `ɵelementEnd`, `ɵlistener`, etc. — this is where DOM nodes are actually created and appended, and where the flat data structures (`LView`, `TView`) that back the view get their static shape.
- **Update block** (`flags & RenderFlags.Update`): runs on every change-detection pass. Emits `ɵadvance(n)` (move the "current node" cursor forward by `n` slots in `LView`) followed by binding instructions like `ɵproperty`, `ɵattribute`, `ɵclassProp`, `ɵtextInterpolate1`, etc. Each binding instruction internally does the dirty-check (`bindingupdated`) against the previous value stored in `LView` and only writes to the real DOM (`Renderer2/native`) if it changed.

Because both blocks are just a flat, ordered list of function calls corresponding 1:1 to the static structure of the template, there is no reconciliation/diffing algorithm needed at all for structural stability — the position in the instruction stream *is* the identity of a node. This is what "incremental DOM-inspired" means precisely: state is not a separate persistent tree object graph (as in a vdom), it is the arrays (`LView` / `TView`) implicitly indexed by instruction call order.

2.4 `ɵdefineComponent` / `ɵdefineDirective` / `ɵdefinePipe`

Every Angular class decorated with `@Component`, `@Directive`, or `@Pipe` gets a **static property** attached to the class by the compiler:

- `@Component` → static `ɵcmp` = the return value of `ɵdefineComponent({...})`
- `@Directive` → static `ɵdir` = the return value of `ɵdefineDirective({...})`
- `@Pipe` → static `ɵpipe` = the return value of `ɵdefinePipe({...})`
- Also always present: `ɵfac` (a `ɵdefineFactory` / `ɵgetInheritedFactory`-generated factory function)

describing how to *construct* the class — i.e., its DI recipe), and for injectables, `@prov`.

These are **plain object literals with a stable, framework-owned shape (a**

`ComponentDef / DirectiveDef / PipeDef`) — not classes, not decorator metadata reflected at runtime. The framework's runtime (`core` package) reads these static properties directly off the class reference whenever it needs to instantiate/render something, e.g., `SomeComponent.ɵcmp.template`, `SomeComponent.ɵcmp.factory`, `SomeComponent.ɵcmp.inputs`.

Key fields on a `ComponentDef`:

Field	Purpose
<code>type</code>	back-reference to the class itself
<code>selectors</code>	parsed selector matcher data (for directive matching against DOM)
<code>factory</code> (from <code>ɵfac</code> , referenced as <code>type.ɵfac</code>)	constructs an instance, resolving constructor DI
<code>template</code>	the create/update instruction function described in §2.3
<code>consts</code> / <code>vars</code>	counts used to pre-size <code>LView</code> /const arrays
<code>hostBindings</code>	instruction function for the component's <i>own</i> host attribute/listener/class/style bindings
<code>inputs</code> / <code>outputs</code>	maps of public binding name → (private class property name, transform fn)
<code>directiveDefs</code> / <code>pipeDefs</code> (or <code>dependencies</code> for standalone)	which other directives/pipes this template's instructions may need to match against — this is the "local resolution table" from §2.2
<code>styles</code> / <code>encapsulation</code>	compiled/scoped CSS, and view encapsulation mode
<code>changeDetection</code>	<code>Default</code> or <code>OnPush</code>
<code>exportAs</code>	template reference variable name(s)

Because this is a **static property assignment**, not a decorator-driven runtime reflection call, bundlers can statically see "this call to `ɵɵdefineComponent` closes over these specific imports (this pipe def, that directive def) and nothing else" and tree-shake anything not actually referenced by any reachable `ɵɵdefineX` call graph.

2.5 Tree-shakability, mechanically

Tree-shaking works because of two combined facts:

1. Angular's own runtime (`@angular/core`) is written so that features are **opt-in through instruction imports**, not through a monolithic "Angular runtime" object. If a template never calls `ɵɵclassMap`, the styling diffing algorithm behind class bindings is never imported. If a component's `changeDetection` is never `OnPush`-relevant to some diffing helper, that helper isn't referenced.
2. `ɵɵdefineComponent`'s `directiveDefs` / `pipeDefs` reference other components' `ɵcmp` / `ɵdir` / `ɵpipe`

objects **directly by JS reference** (an array literal or arrow function returning an array), not by string name lookup through an `NgModule`. A bundler doing standard dead-code elimination on ES modules can already answer "is this export used anywhere reachable from main," and Ivy's output is shaped so that the answer is almost always accurate — an unused component really does have zero remaining references once its declaring module/import is unused, because there's no central "module registry" object holding a reference to every declared component regardless of whether the app uses it (View Engine's `NgModuleFactory` effectively was such an object).

This is why Ivy enabled meaningfully smaller production bundles for the same app: unused Material components, unused pipes, and unused directive input logic no longer had a forced reference path keeping them alive.

2.6 The i18n instruction set

Ivy represents internationalization as its own dedicated instructions rather than a template-preprocessing hack:

- `i18nStart` / `i18n` — marks a block of the create instructions whose text/attribute content should be swapped for a localized message at runtime, keyed by a message id.
- `i18nEnd` — closes the block.
- `i18nAttributes` — applies translated values to element attributes/inputs (e.g., `title`, `aria-label`) separately from text content, since attributes bind differently from text nodes.
- `i18nExp` / `i18nApply` — handle interpolated expressions *inside* a translated string (e.g., `Hello {{name}}`), re-inserting the live expression value into the correct placeholder position of whichever locale's translated string is active, on every update pass.
- `i18nPostprocess` — handles ICU expansion (plural/select) post-processing so `{count, plural, =0 {no items} other {{{count}} items}}`-style ICU blocks resolve correctly per locale.

The key design point: translations are resolved by looking up a `$localize`-tagged message id in a locale data table **at runtime-or-build-time depending on strategy** (Ivy supports both the classic build-per-locale approach and, via `$localize`, a "translate at runtime by swapping global message maps before bootstrap" approach). Because this is expressed as ordinary instructions in the same create/update instruction stream as everything else, i18n content participates in the same flat `LView` change-detection model as regular bindings — no separate i18n-specific rendering pass is needed, and (per §2.5) apps with zero i18n usage pull in none of this code.

2.7 Side-by-side: View Engine vs Ivy

Aspect	View Engine	Ivy
Compilation unit	Whole <code>NgModule</code> graph (global)	Single component/directive/pipe (local)
Generated artifact	<code>*.ngfactory.js</code> , separate <code>view_X</code> render classes	Static <code>ɵcmp</code> / <code>ɵdir</code> / <code>ɵpipe</code> / <code>ɵfac</code> props on the class itself
Dynamic component creation	Requires <code>entryComponents</code>	Works directly via class reference, no registration

Aspect	View Engine	Ivy
Tree-shaking	Poor — module factory graph keeps everything reachable	Strong — per-symbol reference tracing
Rendering model	Interpreted bindings via generated <code>detectChangesInternal</code> methods walking a view-class hierarchy	Flat instruction stream (create/update blocks) over <code>LView</code> / <code>TView</code> arrays
Debug output	Synthetic view classes, poor stack traces	Instructions map closely to template source; <code>ngDevMode</code> -gated helpful errors
Metadata resolution	Needed the declaring <code>NgModule</code> to resolve template element→component matches	Directive/pipe matching table baked directly into the component's own def
i18n	Preprocessing-based	First-class instruction set (<code>ɵi18nStart</code> etc.)
Library distribution	Full AOT-compiled <code>.ngfactory</code> shipped or JIT metadata + <code>.metadata.json</code>	Ivy "partial compilation" output (<code>.ngfactory</code> -free), linked by consumer's compiler
Build incrementality	Poor (global re-resolution)	Good (per-file)
Bundle size (typical)	Larger	Smaller, sometimes dramatically for large component libraries

3. Code Examples

The following is a hand-written approximation of what `ngtsc` (the Ivy AOT compiler) emits for a intentionally trivial component. Real compiler output has extra minified var names, more null-checks, and locale-specific formatting, but this captures the actual shape and the actual instructions used.

Source component:

```
// greeting.component.ts
import { Component, Input, ChangeDetectionStrategy } from '@angular/core';

@Component({
  selector: 'app-greeting',
  template: `
    <div class="card" [class.active]="active">
      <h2>Hello, {{ name }}!</h2>
      <button (click)="onClick()">Say hi</button>
    </div>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class GreetingComponent {
```

```

@Input() name = 'world';
@Input() active = false;

onClick() {
  console.log(`Hi, ${this.name}`);
}
}

```

Simplified compiler output (what actually gets attached to the class):

```

import {
  ɵɵdefineComponent,
  ɵɵdefineFactory,
  ɵɵelementStart,
  ɵɵelementEnd,
  ɵɵtext,
  ɵɵtextInterpolate1,
  ɵɵlistener,
  ɵɵadvance,
  ɵɵclassProp,
  ɵɵproperty,
  RenderFlags,
} from '@angular/core';

class GreetingComponent {
  name = 'world';
  active = false;

  onClick() {
    console.log(`Hi, ${this.name}`);
  }

  // --- Everything below this line is compiler-generated ---

  /** DI recipe: how to construct an instance. No constructor args here, so trivial. */
  static ɵfac = function GreetingComponent_Factory(t) {
    return new (t || GreetingComponent)();
  };

  /** The component definition object read by the runtime at render time. */
  static ɵcmp = ɵɵdefineComponent({
    type: GreetingComponent,
    selectors: [['app-greeting']],
    inputs: {
      name: 'name',
      active: 'active',
    },
    outputs: {},
    decls: 5, // number of "slots" (nodes/bindings) to reserve in LView
    vars: 3, // number of update-block binding slots
    consts: [['class', 'card'], [3, 'click']], // shared static attribute arrays (deduped)

    // ---- CREATE block: runs once, builds the DOM skeleton ----

```

```
// ---- UPDATE block: runs every CD pass, patches bindings ----
template: function GreetingComponent_Template(rf: RenderFlags, ctx: GreetingComponent) {
  if (rf & RenderFlags.Create) {
    eelementStart(0, 'div', 0);      // <div class="card">
    eelementStart(1, 'h2');          // <h2>
    eotext(2);                       // {{ text node, filled in update block }}
    eelementEnd();                   // </h2>
    eelementStart(3, 'button', 1);   // <button (click)="...">
    eolister('click', function
GreetingComponent_Template_button_click_3_listener() {
      return ctx.onClick();
    });
    eotext(4, 'Say hi');             // "Say hi"
    eelementEnd();                   // </button>
    eelementEnd();                   // </div>
  }
  if (rf & RenderFlags.Update) {
    eclassProp('active', ctx.active); // patch [class.active] on node 0
    eadvance(2);                      // move cursor to node index 2 (the text
node)
    eotextInterpolate1('Hello, ', ctx.name, '!'); // patch interpolated text
  }
},

  changeDetection: 0, // 0 = OnPush (ChangeDetectionStrategy.OnPush)
  encapsulation: 2,   // 2 = Emulated (default ViewEncapsulation)
});
}
```

Points to call out when explaining this to an interviewer:

- `decls / vars` pre-size the flat `LView` array so no dynamic array growth happens during rendering — this is a performance detail unique to the flat-array model.
- `consts` holds static, shareable attribute-name arrays (`['class', 'card']`) so multiple instances of the component don't re-allocate identical literals; this is a form of the same "hoist static data out of the per-render hot path" idea used throughout Ivy.
- The **listener closure captures** `ctx` (the component instance passed into the template function), not `this` — this is why Ivy templates don't need `.bind(this)` gymnastics; the runtime always passes the correct context object explicitly.
- `eadvance(2)` is the "cursor" mechanic underpinning the incremental-DOM idea: rather than re-passing a node reference to every update instruction, the runtime keeps an internal pointer into `LView` and each instruction call moves it forward by a known, template-static offset.
- Note there's no `NgModule` anywhere in this compiled output — `selectors` and `inputs / outputs` are entirely sufficient for the runtime to match this component against a parent template's `<app-greeting>` tag and bind to it, which is the locality principle made concrete.

4. Internal Working

4.1 Per-component compilation vs whole-module compilation

View Engine's compiler (`@angular/compiler` driven by the old `ngc`) operated in phases that required a **global metadata collection pass**: read every `@NgModule`, build a map of which components/directives/pipes are in scope for every other component (by walking `declarations` / `imports` / `exports`), *then* emit factories. This is why a single component's factory could not be produced without first resolving the whole module graph it lived in — the compiler literally needed the global answer to "what can `<foo-bar>` in this template possibly refer to" before it could emit code for that template.

Ivy's compiler (`ngtsc`, replacing the old `ngc` /View Engine compiler, built as a TypeScript transform) instead resolves directive/pipe matching **using the component's own decorator metadata**, augmented (for `NgModule`-based apps) by the compiler statically analyzing the `imports` / `declarations` of the *specific* module the component is declared in, but critically it does this by directly embedding the resolved list of candidate directive/pipe defs into that one component's own `@ɵdefineComponent` call as `directiveDefs` / `pipeDefs` — the result is baked into the component's compiled artifact once, at that component's compile time, and never needs re-resolving from the outside afterward. (Standalone components made this even more literal: a component lists its `imports` directly on itself, with zero module indirection at all.)

The practical effect on tree-shaking: a bundler doesn't need Angular-specific knowledge to shake an Ivy app. It just needs to know "is this JS symbol imported from somewhere reachable." Because `@ɵdefineComponent`'s `directiveDefs` is (for eagerly-compiled cases) a literal array of class references or (lazily) an arrow function `() => [OtherDirective, SomePipe]`, standard ES module reachability analysis is sufficient. View Engine's `NgModuleFactory`, by contrast, held a registration table keyed by design (its whole job was "resolve anything declared in this module"), so *nothing* declared in an eagerly-loaded module could easily be proven dead by generic tooling.

4.2 The create/update split as the update algorithm

There is no separate "diff old tree vs new tree" step in Ivy. Change detection for a component means: call `ɵcmp.template(RenderFlags.Update, componentInstance)`. That function is the *same* function used at creation, just with the `update` bit set instead of `Create`, so only the second `if` block executes. Every instruction inside that block (`ɵɵproperty`, `ɵɵattribute`, `ɵɵtextInterpolate*`, `ɵɵclassProp`, `ɵɵstyleProp`, ...) is individually responsible for:

1. Reading the current value of the bound expression (already evaluated by the generated code, passed as an argument).
2. Comparing it to the previous value for *that exact binding slot*, stored in `LView` at a fixed index (dirty-check via `bindingUpdated` / `bindingUpdated2` /... helpers).
3. Writing to the real DOM/renderer *only if changed*, then storing the new value back into `LView` for next time.

Because slot indices are static per template (determined at compile time from the order instructions appear in the create block), there's no need for key-based reconciliation, no need to allocate a new tree per render, and no need for a "commit phase" separate from the "render phase" — writes happen instruction-by-instruction, immediately, as soon as a changed value is detected. This is the literal meaning of "incremental DOM": you incrementally patch the *existing* DOM tree in place by re-running the same deterministic instruction sequence, rather than building a *new* virtual tree and diffing it against the old one.

`TView` (shared across all instances of a component, created once) stores the static template shape (which nodes exist, their attribute const references, `firstCreatePass`-only data); `LView` (one per instance, and one per embedded/host view) stores the actual live data (DOM node references, directive instances, bound values, the component context). Splitting static-shared (`TView`) from per-instance (`LView`) data is itself a tree-shaking/perf-adjacent optimization: it avoids re-deriving the same static template metadata for every instance of a repeated component (e.g., every row in an `*ngFor`).

4.3 Build pipeline difference

- View Engine: TypeScript compiles `.ts` → `.js`; separately, the Angular template compiler reads `.metadata.json` (JIT) or runs `ngc` as a wrapping compiler (AOT) to emit `.ngfactory.ts` files that are *then* compiled by TypeScript in a second pass. Two effectively sequential, whole-program-aware compiler invocations.
- Ivy: `ngtsc` is a **TypeScript compiler plugin** (a `ts.CompilerHost` /transform wrapper) that runs template compilation *inline* with normal TypeScript compilation, emitting the `ɵdefineComponent(...)` static property directly into the same `.js` output file TypeScript would have produced anyway — no separate `.ngfactory.ts` file, no second pass. This is a major reason Ivy AOT is close in speed to plain `tsc`, and why AOT became the default build mode for `ng build / ng serve` from Angular 9 onward (previously AOT was reserved for production builds due to its cost).

5. Edge Cases & Gotchas

- **ngcc (Angular Compatibility Compiler).** Libraries published *before* Ivy shipped their compiled output in View-Engine format (`.metadata.json` + partially-compiled `.js` referencing the old `ViewEngine` factory shape). Since an Ivy app cannot consume View Engine factories directly, Angular CLI ran `ngcc` as a postinstall step: it scanned `node_modules`, found View-Engine-shaped Angular packages, and **compiled them in place** into Ivy-shaped `ɵdefineComponent` output, caching the result so it only re-ran when packages changed. This was transparent but could be slow on first install and was a common source of "why is `ng serve` slow the first time" confusion. `ngcc` was removed in Angular 16+ once the ecosystem had fully migrated to Ivy-native ("partial compilation") package distribution.
- **Partial compilation for libraries.** Since Ivy, `ng-packagr` /the Angular compiler emits libraries in a **partial-ivy format**: instructions like `ɵngDeclareComponent` (a versioned, higher-level IR) instead of the raw low-level `ɵdefineComponent` instruction stream. The *consuming* application's own build then "links" this partial format down to full low-level instructions matching that app's exact Angular runtime version. This exists because raw Ivy instructions are **not considered a stable public API** across Angular versions (deliberately — it lets the Ivy runtime evolve instruction shapes between majors), whereas the partial/declarative format is stable and versioned, so a library built against Angular 14 can still be consumed by an Angular 17 app without recompilation from source.
- **The "Ivy instruction set is not public API" trap.** Some early adopters wrote code or tooling that inspected `ɵcmp / ɵdefineComponent` output directly (e.g., custom testing utilities, meta-frameworks). Because these are prefixed `ɵ` (a convention Angular uses for "private, may change without a major version bump, do not depend on this"), such code broke across minor/patch Ivy releases. The only supported public surface is decorators, `NgModule` /standalone metadata, DI tokens, and documented `@angular/core` exports — never the `ɵ`-prefixed static properties.
- **Dynamic component creation semantics changed subtly.** `ComponentFactoryResolver / entryComponents` became unnecessary and were eventually

deprecated/removed because Ivy can synthesize what it needs from `SomeComponent.ɵcmp` directly (`ViewContainerRef.createComponent(SomeComponent)` without a factory argument). Code written for View Engine that manually resolved `ComponentFactory` via a `ComponentFactoryResolver` still worked (Ivy provided a compatibility shim producing equivalent factory-like objects), but new Ivy-only APIs (like `createComponent` with an `environmentInjector`) bypass that layer entirely and are the encouraged path — mixing old-style factory-resolver code with new-style direct creation in the same app is a common "which pattern do I follow" confusion point in migrated codebases.

- **NgModule-declared vs standalone directive matching are not the same code path historically.** Early Ivy releases resolved a template's matchable directives/pipes only from the enclosing `NgModule`'s compiled scope; if a component was accidentally declared in two modules, or a shared module didn't re-export a needed directive, template matching silently failed at runtime with "not a known element" — a class of bug that didn't exist the same way under View Engine's more eagerly-validated global graph (though View Engine had its own equally painful "no NgModule metadata found" failures). Migrating large module-based apps, this showed up as previously-working templates suddenly failing to bind custom attribute directives if the module wiring had latent bugs View Engine happened not to trip over.
- **AOT became mandatory-by-default; JIT-only edge cases surfaced.** Since Ivy made AOT fast enough to use in dev mode too, JIT compilation (`@angular/compiler` at runtime, used e.g., by `TestBed` in older configurations or truly dynamic runtime-defined components) became the less-traveled path. Some patterns that worked fine in View Engine JIT — like defining a component's `template` string dynamically at runtime — needed the JIT compiler explicitly loaded and could produce confusing "the injectable component compiler is not available, please add `@angular/compiler` to your polyfills" errors in Ivy builds that assumed AOT-only.
- **Style encapsulation edge cases in migrated hybrid apps.** Ivy's emulated encapsulation attribute-scoping (`_ngcontent-xxx / _ngghost-xxx`) mechanism is compatible in spirit with View Engine's but not byte-identical; apps mid-migration that mixed Ivy-compiled and (temporarily, via `ngcc`-shimmed) View-Engine-compiled component styles occasionally saw subtly different CSS specificity-selector generation, requiring visual QA during the View Engine → Ivy migration window.
- **fullTemplateTypeCheck / strict template type-checking got much stronger under Ivy** (`strictTemplates` in `angular.json`). Templates that silently compiled under View Engine's looser type checking (implicit `any` in many binding expressions) frequently produced **new, legitimate compile errors** the first time a codebase turned on Ivy's stricter default type-checker — a very common "why did my build suddenly break after the Ivy upgrade" support question, whose real answer was "the old checker wasn't actually checking this."

6. Interview Questions & Answers

Q1. In one sentence, what is Ivy?

A: Ivy is Angular's compiler-and-runtime architecture (default since Angular 9) that compiles each component/directive/pipe independently into a small set of imperative instruction functions and a static metadata object on the class, replacing View Engine's whole-module `NgFactory` generation model.

Q2. What concrete problem in View Engine did Ivy's "locality principle" fix?

A: View Engine needed the *entire* `NgModule` graph resolved before it could emit a factory for even one component, because template element-to-component matching was resolved at the module level, not the component level. That forced global, whole-program compilation, which in turn defeated tree-shaking (an `NgModuleFactory` held references to everything it might ever need) and slowed incremental rebuilds. Locality means a component's compiled output is self-sufficient — computed from that component's own metadata

alone — so compiling/recompiling one file never requires touching others, enabling per-file incrementality and reference-based tree-shaking.

Interviewer intent: Checking that the candidate understands locality as an architectural constraint with causal downstream effects, not just a buzzword — many candidates can say "Ivy is tree-shakable" without being able to explain the actual compilation-model change that causes it.

Q3. What are `@defineComponent`, `@defineDirective`, and `@definePipe`, and where do they live on the class?

A: They are compiler-generated calls, invoked once per class, whose return value is assigned to a static property on the decorated class: `@cmp` for components, `@dir` for directives, `@pipe` for pipes (plus `@fac` for the DI factory on all of them). Each call takes an object literal describing selectors, inputs/outputs, the create/update template instruction function (components only), host bindings, change detection strategy, encapsulation, and (for components) the list of directive/pipe defs the template may match against. The Angular runtime reads these static properties directly off the class reference at render/instantiation time instead of doing any decorator reflection.

Q4. Why did `entryComponents` disappear in Ivy?

A: `entryComponents` existed so View Engine's global compiler would know, ahead of time, to generate a `ComponentFactory` for components only ever created dynamically (never referenced directly in any template), since otherwise nothing would trigger their compilation into the module's factory graph. Under Ivy, a component's `@cmp` definition is produced the moment that one file is compiled, regardless of whether anything else references it in a template — so `ViewContainerRef.createComponent(SomeComponent)` can synthesize what it needs directly from `SomeComponent.@cmp` at call time, with no advance registration required.

Q5. Describe the "create block" / "update block" split in a compiled Ivy template function.

A: Every component's `template` function is called with a `RenderFlags` bitmask and the component instance. On first render, `RenderFlags.Create` is set, and only the create-block `if` branch runs — it emits instructions like `@elementStart` / `@text` / `@listener` that build the actual DOM structure and register event handlers, once. On every subsequent change-detection run, `RenderFlags.Update` is set and only the update-block branch runs — instructions like `@property` / `@classProp` / `@textInterpolate1` re-evaluate bound expressions and patch the DOM if values changed. The same function, and the same instruction ordering, is reused for both — there's no separate "diffing" data structure; position in the instruction stream is stable and implicit.

Q6. How does Ivy's rendering model relate to "incremental DOM," and how does that differ from virtual DOM?

A: Virtual DOM libraries build a new lightweight tree object on every render and diff it against the previous tree to compute a patch — this costs an allocation and a comparison pass over the whole tree each render, regardless of what actually changed. Incremental DOM (the approach Ivy's authors drew on) instead compiles the template into instructions that directly walk/patch the *real* DOM in a fixed, deterministic order; each instruction knows its own previous value (stored in a flat per-instance array, `LView`) and skips work if nothing changed. No parallel tree data structure is ever allocated per render — the "old tree" is just the real DOM plus the previous values already sitting in `LView`.

Interviewer intent: Distinguishing candidates who've memorized "Ivy uses incremental DOM" from those who can actually explain what problem that avoids (double allocation, need for reconciliation/keying) versus vdom frameworks.

Q7. What are `LView` and `TView`, and why are they split?

A: `TView` ("template view") holds data that's static and shared across every instance of a given template — the compiled instruction function reference, static const arrays, binding count, first-create-pass-only bookkeeping. `LView` ("live view" / "logical view") holds per-instance data — actual DOM node references, directive instances, current bound values, the component context object, and a back-pointer to its `TView`. They're split so that repeated views (e.g., every row rendered by an `*ngFor`, or every instance of a reused component) share one `TView` computed once, and only allocate the smaller, per-instance `LView` for each repetition — avoiding redundant recomputation of static template metadata.

Q8. Why is `directiveDefs / pipeDefs` on `@defineComponent` important for tree-shaking?

A: Because it's a literal array of direct JS references (or a lazily-evaluated arrow function returning such an array) to other components'/pipes' `@dir / @cmp / @pipe` objects, rather than a lookup by string name through some central module registry. A bundler's standard ES-module reachability/dead-code-elimination pass can trace "is `SomePipe` reachable from `main.ts`" the same way it would for any ordinary JS import — no Angular-specific knowledge required. If nothing in the reachable graph imports a given component/pipe, it's provably unused and gets dropped, unlike View Engine's `NgModuleFactory`, which held a registration table keyed to resolve *anything declared in the module*, keeping everything reachable regardless of actual usage.

Q9. What is `ngcc`, why did it exist, and why was it removed?

A: `ngcc` (Angular Compatibility Compiler) rewrote pre-Ivy, View-Engine-format compiled libraries in `node_modules` into Ivy-compatible output, since an Ivy application cannot consume raw View Engine `.metadata.json`/factory-shaped code. It ran as a build/postinstall step, cached its output, and was necessary during the migration window while much of the ecosystem (npm packages) hadn't yet republished Ivy-native builds. It was removed starting Angular 16 once the ecosystem had converged on Ivy's "partial compilation" distribution format for libraries, making the compatibility rewrite step unnecessary.

Q10. What is "partial compilation," and why isn't the raw Ivy instruction set used directly for published libraries?

A: Partial compilation is an intermediate, versioned, declarative IR (instructions like `@ngDeclareComponent`) that `ng-packagr`/the Angular compiler emits for published libraries instead of the raw low-level instruction stream (`@elementStart`, `@property`, etc.). The raw instruction set is deliberately treated as a private, unstable implementation detail that can change shape between Angular versions to let the runtime evolve; if libraries shipped raw instructions, every library would break whenever Angular's internal instruction shapes changed. The partial format is stable/versioned; the consuming application's own toolchain "links" it down to the exact low-level instructions matching whatever Angular version that app is actually running, at that app's build time.

Q11. A team migrates a large `NgModule`-based app from View Engine to Ivy and suddenly gets new compile errors about template type mismatches that never appeared before. Why?

A: Ivy ships with a substantially stricter template type-checker (`strictTemplates`, and generally tighter default `fullTemplateTypeCheck` behavior) than View Engine's looser checking, which frequently left binding expressions effectively typed as `any`. Enabling Ivy (or subsequently opting into `strictTemplates: true`) surfaces genuine, pre-existing type errors in templates that the old checker silently let through — it's not new breakage introduced by Ivy, it's latent bugs the old compiler failed to catch. The fix is to actually correct the underlying type mismatches (or, as a stopgap, dial back `strictTemplates`), not to treat it as an Ivy regression.

Interviewer intent: Tests whether the candidate has actually lived through a View Engine → Ivy migration and understands that "new errors after upgrade" usually means "better detection," a common real-world gotcha versus a purely theoretical understanding of Ivy.

Q12. How does Ivy achieve faster incremental (re)compilation than View Engine, mechanically, not just "it's local"?

A: `ngtsc` is implemented as a transform integrated directly into the TypeScript compiler's own program/incremental-build machinery, rather than a wrapping second-pass compiler that generates intermediate `.ngfactory.ts` files needing a further `tsc` pass (View Engine's model). Because each component's `@defineComponent` call can be emitted from that file's own decorator + template alone (locality), TypeScript's existing incremental compilation (`--incremental` /project references, or the CLI's own build cache) can treat "recompile this one changed `.ts` file" as sufficient — there's no need to re-resolve a global `NgModule` metadata graph to determine what changed. This lets Ivy AOT compilation run fast enough to be Angular's default even for `ng serve` dev rebuilds, whereas View Engine AOT was historically reserved for production builds due to its cost.

Q13. Does Ivy change how change detection decides *when* to check a component (zones, `OnPush` gating), or only *how* the check is executed once triggered?

A: Ivy changes the *mechanics of execution* — how a component's bindings are actually re-evaluated and patched to the DOM (flat instruction re-run over `LView` versus View Engine's generated `detectChangesInternal` methods) — but the *triggering* model (Zone.js patches async APIs and calls `ApplicationRef.tick()`, which walks the component tree respecting `OnPush` / `markForCheck` / dirty flags) is conceptually the same contract Angular has had since View Engine; Ivy did not itself introduce zoneless change detection or signals (those came later, as Ivy-only features built once locality/instruction-based rendering made them tractable). The interview-safe way to phrase it: Ivy is a renderer/compiler rewrite, and it's also the architectural foundation that made later reactivity changes (signals, zoneless CD) feasible, but Ivy itself did not replace the Zone-based triggering model.

Q14. What replaced `ComponentFactoryResolver` in modern Ivy-era Angular, and why was the old API kept working during the transition?

A: The modern path is `ViewContainerRef.createComponent(Component, options)` (and the standalone `createComponent()` function with an `EnvironmentInjector`), which take a component class reference directly and synthesize what's needed from its `@cmp` at call time — no factory resolution step at all. `ComponentFactoryResolver` / `ComponentFactory` were kept functioning (backed by shim logic producing Ivy-derived factory-like objects) for a long deprecation window so that existing library and application code performing dynamic component creation didn't break immediately on upgrading to Ivy; they were formally deprecated and later removed only once the ecosystem had had several majors to migrate off them, consistent with Angular's general policy of deprecate-then-remove across major versions rather than breaking synchronously with an internal rendering rewrite.

7. Quick Revision Cheat Sheet

- **Ivy** = Angular's default compiler/renderer since v9; replaced **View Engine**.
- **Locality principle**: a component's compiled output depends only on its own decorator + template — never on other files/modules. This is the root cause of every other Ivy benefit.
- **View Engine problem**: whole-`NgModule`-graph (global) compilation → poor tree-shaking, slow incremental builds, `entryComponents` required for dynamic creation, synthetic/unhelpful stack traces.
- **Static defs**: `@cmp` (`@defineComponent`), `@dir` (`@defineDirective`), `@pipe` (`@definePipe`), `@fac` (factory/DI recipe) — plain object literals attached as static class properties, read directly by the runtime.
- **Instruction-based rendering**: template compiles to a `template(rf, ctx)` function with a **create**

block (`RenderFlags.Create`, runs once — builds DOM) and an **update block** (`RenderFlags.Update`, runs every CD pass — patches bindings). Same function, same call, flag decides which half runs.

- **"Incremental DOM" inspiration:** no vdom tree is ever allocated; instructions patch the real DOM directly, using position-in-instruction-stream as implicit node identity, and previous values are cached per-slot in `LView` for dirty-checking.
- `TView` = static, shared-per-template data (compiled once). `LView` = per-instance live data (DOM refs, bound values, context). Split avoids recomputation across repeated instances (e.g., `*ngFor` rows).
- **Tree-shaking mechanism:** `directiveDefs` / `pipeDefs` are direct JS references (not string/module-registry lookups) → standard bundler dead-code elimination just works, unlike View Engine's `NgModuleFactory` registry that kept everything in a module reachable.
- **entryComponents is gone:** Ivy synthesizes what it needs from a class's own `ɵcmp` at dynamic-creation time; no advance registration needed.
- **i18n instructions:** `ɵi18nStart` / `ɵi18n` / `ɵi18nEnd` (block boundaries), `ɵi18nAttributes` (translated attributes), `ɵi18nExp` / `ɵi18nApply` (interpolated expressions inside translations), `ɵi18nPostprocess` (ICU plural/select). First-class instructions, not a preprocessing hack — participate in normal CD.
- **ngcc:** migration-era tool that rewrote View-Engine-format `node_modules` packages into Ivy format; removed in Angular 16+ once libraries ship Ivy-native.
- **Partial compilation:** libraries ship a stable, versioned declarative IR (`ɵɵngDeclareComponent`, etc.); the consuming app's compiler "links" it to full low-level instructions — because raw Ivy instructions are private/unstable (`ɵ` prefix) and may change between Angular versions.
- **Migration gotchas:** stricter default template type-checking surfaces previously-hidden template bugs; `ɵ`-prefixed internals are not public API and can break across versions; JIT-only dynamic-template patterns need `@angular/compiler` explicitly available; mixed View-Engine/Ivy style-encapsulation output during migration windows needs visual QA.

Created By - Durgesh Singh